Documentation - Team 15

Digital module

Our digital module involves two submodules: device running Mr. Kip that can be tilted (smartphone seems ideal for this) and a device with a screen on which the user interface is displayed.

Device with Mr. Kip

Requirements:

- easy to tilt
- Internet connection (to connect to OOCSI server)

The project was prepared with default variant of Mr Kip in mind, but any modified variant of Mr Kip will work provided that the messages sent via the OOCSI server still include *mrkip_d1* parameter in its original meaning (so horizontal movement change). It is also important to ensure that Mr. Kip publishes the data to the channel to which the other submodule subscribes.

Device with a screen

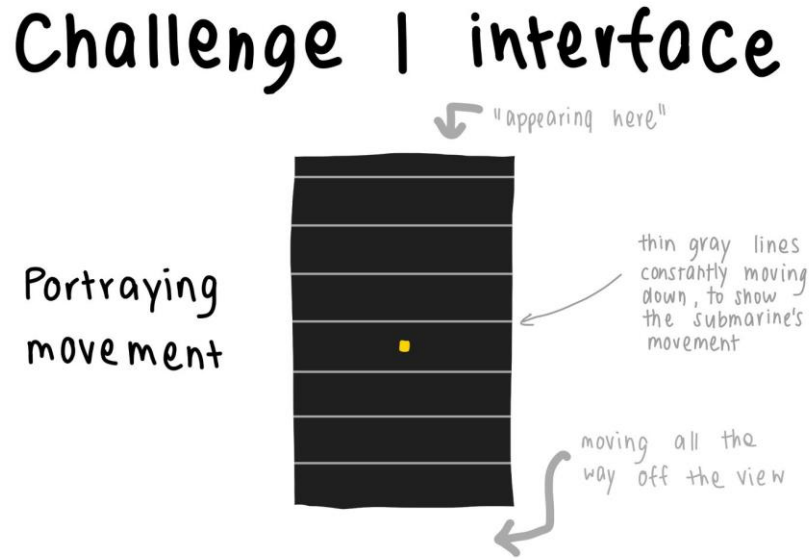
Requirements:

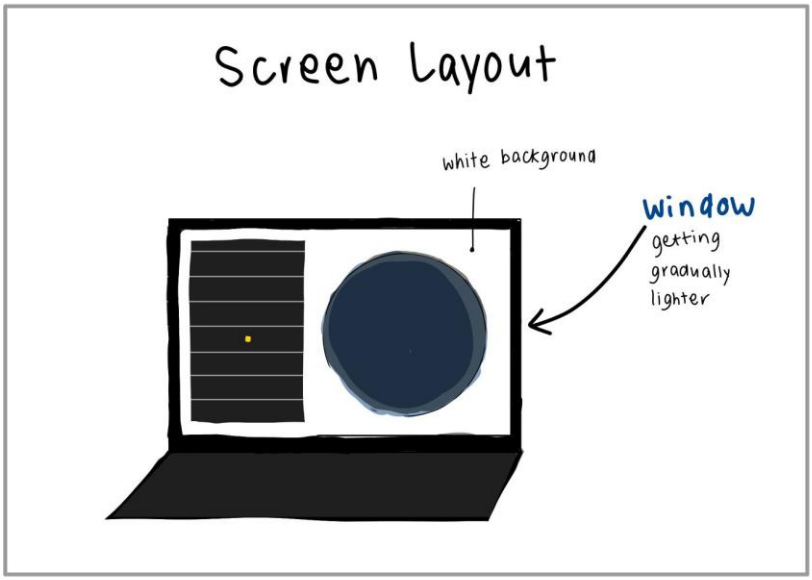
- display such that looking at the UI is comfortable for the player
- Internet connection (to receive and send data via OOCSI server)

The code responsible for this submodule is **challenge1_team.pde**. To improve the flexibility of the code, we decided for great modularization of the code. Moreover, for easier modification, a lot of global constants are put at the beginning of the program to be easily accessible together with comments explaining the constants' roles.

Note: by default, the interface does not show the obstacles, as our intention was to force the players to cooperate by giving one of them control over steering wheel with a view on the interface, while the other player would need to use the physical component to determine how far the obstacles are located. However, if you wish to make obstacles visible, for example for **debugging or testing reasons**, we would like to mention that this feature can be changed by simple changing the boolean value of variable SHOW_OBSTACLES.

Below we present the layout of the user interface:



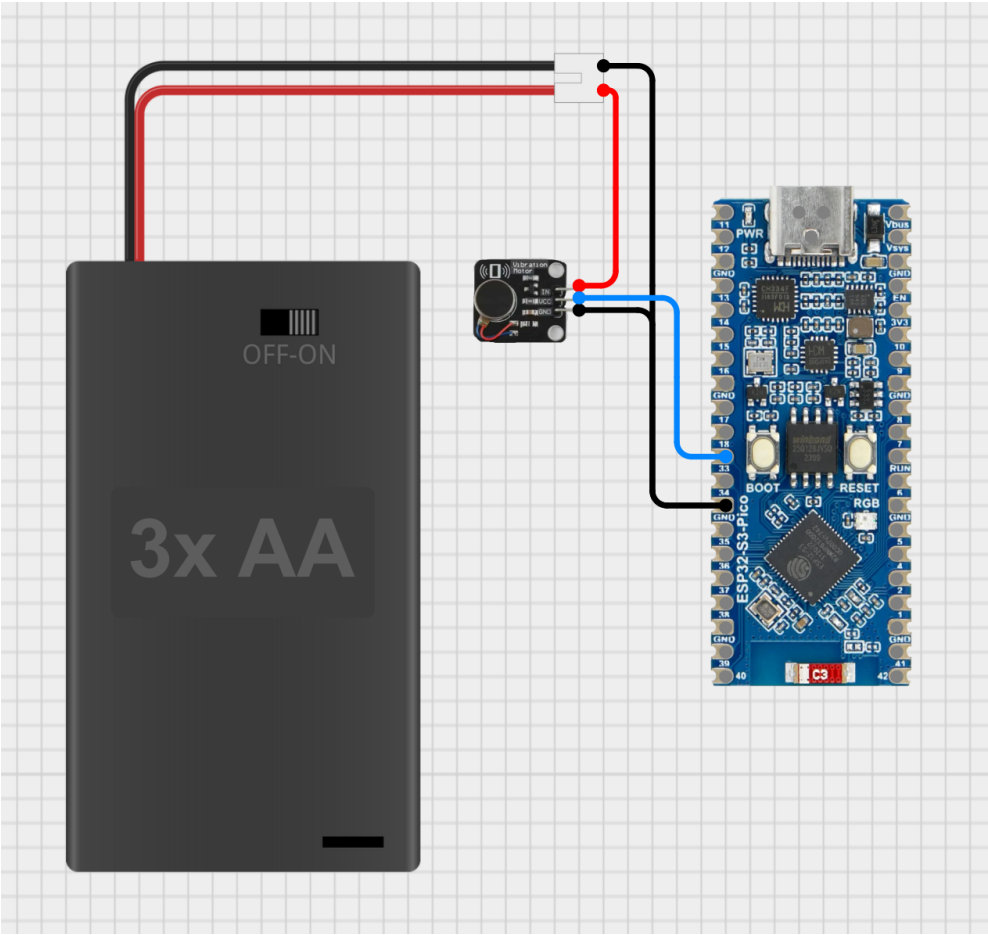


The smallest required change necessary to adopt the module is changing the definition of objects OOCSI_LISTENER (listens to data sent by Mr. Kip) and OOCSI_SENDER (sends the distance data to device) such that the objects have required name and are located within required teamspace, as well as changing the value of variable DISTANCE_CHANNEL_NAME which determines to which channel data are sent.

```
//OOCSI seetings
OOCSI OOCSI_LISTENER = new OOCSI(this, "OOCSI-things/team-15/Kip-listener", "oocsi.id.tue.nl"); //listens to data sent by MrKip
OOCSI OOCSI_SENDER = new OOCSI(this, "OOCSI-things/team-15/Distance_sender", "oocsi.id.tue.nl"); //publishes the distance data
String DISTANCE_CHANNEL_NAME = "OOCSI-things/team-15"; //change if you want to submit to other channel
```

Smallest required change to adopt the module requires changing these 3 lines

Physical module



We initially planned to use a piezo disk to create the vibrations, but it could not be felt, only heard. We chose a DC Vibration motor instead to provide haptic feedback, as it was important that the other player couldn't hear the buzzing to preserve the concept of working together

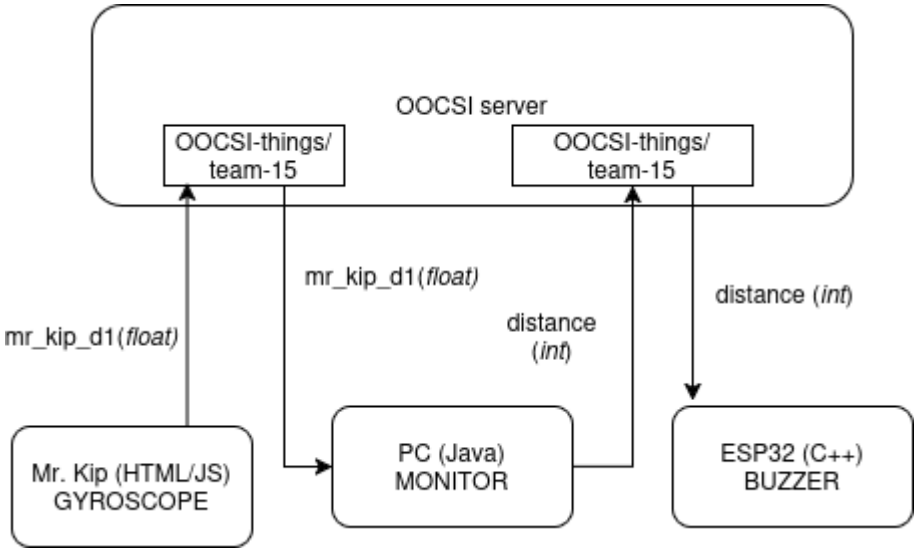
The DC Vibration motor requires a higher voltage than the ESP32 can safely provide. To run it safely and avoid damaging the ESP32 by running too much current, we connected it to 3 AA batteries which supply 4.5V.

The code for the ESP32 is contained in the file OOCSI_BUZZYBEE.ino together with explanatory comments. The minimal required change to adopt the module involves changing the following lines of code. The name of the device will be: "[COURSE]/[TEAM]/[THING]" and the channel to which the ESP32 subscribes is "[COURSE]/[TEAM]".

```
#define SEP "/"
// this is the course space
#define COURSE "OOCSI-things"
// this is the team space
#define TEAM "team-15"
// this is the thing name
#define THING "BUZZYBEE_####"
```

Smallest required change to adopt the module requires changing these definitions

Data exchange



Channel from which Java code receives the data generated by Mr. Kip is determined by OOCSI_LISTENER variable. Channel to which distance measurements are sent is determined by OOCSI_SENDER variable.

DIY elements

In alignment with the submarine theme, we made 2 DIY physical elements:

- Steering wheel setup
- Vibration element

They were put together largely using recycled materials including cardboard, cans, and foam. They were later painted, with details like the logo and a handprint to add to the

unique escape room aesthetic.